FH JOANNEUM - University of Applied Sciences

Pods Are Not Shards: An Empirical Comparison of Kubernetes-Native and Topology-Aware Operator-Based Autoscaling for Stateful Redis Clusters

Bachelorarbeit

zur Erlangung des akademischen Grades

„Bachelor of Science in Engineering“

eingereicht am Studiengang Wirtschaftsinformatik

Studienrichtung: Wirtschaftsinformatik

Verfasst von: Johann Krischan Hipolito

Betreuung: Michael Nestler

Graz, 2026

FH | JOANNEUM
University of Applied Sciences

Table of Contents

List of Figures

List of Tables

List of Abbreviations

API Application Programming Interface

YAML YAML Ain’t Markup Language

KEDA Kubernetes Event-Driven Autoscaling

CRD Custom Resource Definition

HTTP Hypertext Transfer Protocol

HPA Horizontal Pod Autoscalar

JS JavaScript

List of Symbols

Hier steht das Symbolverzeichnis.

List of Formulas

Hier steht das Formelverzeichnis.

Kurzfassung

Hier steht die Kurzfassung auf Deutsch.

Abstract

Here is the abstract in English.

1 Introduction

1.1 Cloud Computing and the Rise of Container Orchestration

The rapid evolution of modern software systems has fundamentally transformed the way infrastructure is designed, deployed, and operated. At the heart of this transformation lies *cloud computing*—a paradigm that enables on-demand access to a shared pool of configurable computing resources, including networks, servers, storage, applications, and services, that can be rapidly provisioned and released with minimal management effort (Mell and Grance, 2011). This model has shifted the industry away from static, on-premises data centres toward elastic, scalable environments that can react dynamically to workload demands.

As organisations increasingly adopt cloud-native architectures, the notion of designing systems that are resilient, scalable, and operationally excellent has become central to engineering practice. The AWS Well-Architected Framework, for instance, codifies these concerns into a set of established design principles for container-based workloads, emphasising repeatability, automation, and observability as first-class concerns in modern infrastructure (Amazon Web Services, 2022).

A critical enabler of cloud-native architectures is the *container*—a lightweight, self-contained unit of software that packages application code together with its dependencies, configuration, and runtime environment. Unlike traditional virtual machines, containers share the host operating system kernel and therefore incur significantly less overhead while still providing process-level isolation (Docker Inc., 2016). This characteristic makes containers especially well suited to microservice architectures, where large monolithic applications are decomposed into smaller, independently deployable services that can be scaled individually based on demand.

1.2 Kubernetes as the Standard for Container Orchestration

Managing containerised workloads at scale introduces significant operational complexity. Scheduling containers across a fleet of machines, maintaining desired application state, performing rolling updates, and recovering from failures all require robust automation. It is in response to exactly these challenges that *Kubernetes*—also referred to as K8s—emerged as the dominant container orchestration platform.

Originally designed and used internally at Google as the *Borg* system, Kubernetes was open-sourced in 2014 and donated to the Cloud Native Computing Foundation (CNCF), where it has since grown into the second-largest open-source project in the world (IBM, 2024). Kubernetes provides a declarative Application Programming Interface (API) for defining the desired state of an application cluster; its control plane continuously reconciles the observed state of the system against this desired state, automatically scheduling pods, managing service discovery, and orchestrating storage (Kubernetes Documentation, 2024b).

To reduce operational complexity further, lightweight Kubernetes distributions such

as *K3s*—developed by Rancher and certified by the CNCF—have gained widespread adoption. K3s packages the full Kubernetes feature set into a single binary with a significantly reduced memory footprint, making it suitable for edge computing, IoT environments, and resource-constrained infrastructure (K3s Project, 2024). In the context of this thesis, K3s serves as the underlying cluster management layer for the experimental infrastructure under investigation.

1.3 The Challenge of Scaling Stateful Applications

While Kubernetes excels at managing stateless workloads—where any replica of a service can handle any request independently—scaling *stateful applications* presents a distinct set of challenges. Stateful applications, such as in-memory databases and message queues, maintain persistent data or internal state that must be carefully managed across the lifecycle of individual pods. Kubernetes provides the `StatefulSet` primitive to address these requirements, offering stable network identities, ordered deployment and scaling, and persistent volume binding (Kubernetes Documentation, 2024d).

Redis is a canonical example of such a stateful application. It is a high-performance, in-memory data store widely used for caching, session management, and real-time analytics (Redis Ltd., 2024a). When deployed as a Redis Cluster across multiple nodes, it uses sharding to distribute its keyspace and replication to ensure fault tolerance. Scaling a Redis Cluster in a Kubernetes environment is non-trivial: adding or removing nodes requires the coordinated resharding of data slots, careful management of cluster topology, and maintenance of quorum—operations that introduce significant risk if performed without regard to the cluster's internal state.

Recent work from the CNCF community has highlighted these concerns, noting that while Kubernetes operators can partially automate lifecycle management for Redis, the path toward robust, cloud-native stateful services remains an open challenge (Cloud Native Computing Foundation, 2024).

1.4 Native Scaling and the Topology Gap

Kubernetes provides native support for horizontal scaling through the *Horizontal Pod Autoscaler* (HPA), which monitors resource-utilisation metrics such as CPU and memory and adjusts the number of pod replicas accordingly (Kubernetes Documentation, 2024a). For stateless services this mechanism is entirely sufficient, because every replica is interchangeable and may therefore be added or removed without coordination. The HPA's reactive nature—scaling decisions are taken only after a utilisation threshold has already been breached—has been criticised as ill-suited to latency-sensitive and stateful workloads (Thoras, 2024), and event-driven frameworks such as Kubernetes Event-Driven Autoscaling (KEDA) have emerged to move the scaling decision upstream of resource exhaustion by triggering on application-level signals (KEDA Project, 2024a; Cloud Native Computing Foundation, 2023).

For a sharded, stateful store, however, the more fundamental difficulty lies not in

when a scaling decision is taken but in *how* it is mechanically carried out. The HPA, and indeed any controller that drives a `StatefulSet`, manipulates only the replica *count*; it possesses no model of the cluster's internal *topology* and, in particular, no notion of which of the $16\,384$ hash slots a given pod owns (Redis Ltd., 2024c). On scale-out this limitation manifests as capacity that the cluster cannot integrate, but on scale-in—the operation with which this thesis is principally concerned—it becomes far more dangerous, because deleting a slot-owning master without first migrating its slots elsewhere abandons a portion of the keyspace outright (Redis, n.d.; Google Cloud, 2023). This is the precise sense in which *pods are not shards*: terminating a pod does not relocate the data for which it was responsible.

It is to close exactly this gap that the Kubernetes *operator pattern* was conceived. An operator encodes the application-specific operational knowledge of a human expert into a controller that participates in the standard reconciliation loop (Kubernetes Documentation, 2024c), and a Redis operator can therefore enforce the invariant that a master must be *drained*—its slots resharded onto the survivors—before it is removed. The central comparison undertaken by this thesis is consequently not one of reactive against predictive autoscaling, but of topology-blind against topology-aware scaling, evaluated on the single operation where the distinction carries the gravest consequences for data.

1.5 Research Objective and Thesis Structure

The title of this thesis—*Pods Are Not Shards*—names the gap identified above: a Kubernetes pod is a unit of *scheduling*, whereas a Redis shard is a unit of *data ownership*, and the two coincide only for as long as the cluster's topology remains undisturbed. This thesis investigates what happens when that topology is disturbed in the most consequential way—by removing a master during a planned scale-in—and poses a single, precise question: does a planned, one-master scale-in of a sharded Redis Cluster preserve all $16\,384$ hash slots together with every stored key, and to what extent does the answer depend on the mechanism that carries the removal out?

To answer this question, the thesis conducts an empirical, controlled comparison of the two mechanisms by which such a removal can occur. The first is the native Kubernetes path, in which a `StatefulSet` is simply contracted—the precise action that the HPA performs on scale-down—deleting a pod by ordinal with no awareness of the hash slots it owns. The second is the operator path, in which the Opstree Redis Operator, having encoded Redis-specific domain knowledge into a Kubernetes controller, drains the departing master by migrating its slots to the survivors before the pod is removed (OpsTree Solutions, n.d.; The Kubernetes Authors, n.d.-b). The dependent variable throughout is *data loss*—the number of keys lost and the number of hash slots left permanently uncovered—rather than the responsiveness or autonomy of any autoscaler.

A deliberate methodological choice underpins the comparison. So that the outcome can be attributed to the scaling mechanism alone, the experiment is conducted under strictly quiescent conditions: a known, exact dataset is written directly into the cluster,

after which exactly one master is removed, with no client load, no application entry-point, and no autoscaler controller in the loop. This isolation eliminates the dominant confound—write pressure interrupting a live reshard—and renders the result deterministic and reproducible. The event-driven triggering of scaling decisions, including the use of KEDA and application-level Prometheus metrics, together with the behaviour of the system under sustained load, are orthogonal to the data-safety question examined here and are accordingly treated as background and as directions for future work rather than as variables under test.

The experimental infrastructure is correspondingly lean. It comprises a Redis Cluster deployed in two interchangeable configurations—a self-managed cluster assembled from native primitives and an operator-managed cluster declared through a custom resource—together with an observability stack of Prometheus and Grafana on which the before-and-after slot and key metrics are captured. All components run on a K3s cluster distributed across resource-constrained mini-PCs, providing a reproducible environment representative of real-world edge and on-premises deployments (K3s Project, 2024).

The remainder of this thesis is structured as follows. Chapter **??** examines the distinction between scaling stateless and stateful workloads and the specific hazards that arise when a sharded store is scaled. Chapter 3 provides the theoretical background on Kubernetes, the `StatefulSet` primitive, the operator pattern, and the Redis Cluster model. Chapter 5 describes the system design and experimental testbed, and Chapter 6 sets out the controlled procedure that exercises it. Chapter 7 presents the experimental results and their analysis, and Chapter 8 summarises the findings and discusses directions for future work.

2 Hypothesis: Topology-Aware Operation as a Precondition for Data-Safe Scale-In

The limitations of topology-blind, count-based scaling described in Section **??** motivate the central hypothesis of this thesis: that *topology-aware operation*—the encoding of Redis-specific resharding logic into a Kubernetes operator—is a precondition for the data-safe scale-in of a sharded, stateful workload, and that the native StatefulSet contraction performed by the Horizontal Pod Autoscaler, being blind to the cluster's slot map, cannot preserve data when a slot-owning master is removed. The hypothesis is therefore framed not in terms of how *quickly* or how *autonomously* a scaling decision is reached, but in terms of whether the scaling *action*, once taken, leaves the cluster's keyspace intact; the governing metric is accordingly *data loss*—keys lost and hash slots left permanently uncovered—rather than responsiveness or resource efficiency.

2.1 The Operator Pattern as a Foundation for Stateful Automation

The Kubernetes *operator pattern* was conceived precisely to bridge the gap between the generic automation that Kubernetes provides out of the box and the deep, application-specific knowledge required to manage complex stateful systems reliably. According to the official Kubernetes documentation, the operator pattern "aims to capture the key aim of a human operator who is managing a service or set of services", encoding that operational expertise into a software controller that participates in the standard reconciliation loop (Kubernetes Documentation, 2024c). Operators extend the Kubernetes API through *Custom Resource Definitions* (CRDs) together with an accompanying controller that continuously reconciles the observed state of the custom resource against its declared desired state.

Critically, the CNCF Operator White Paper explicitly acknowledges that "Kubernetes primitives were not built to manage state by default" and that "relying on Kubernetes primitives alone brings difficulty managing stateful application" lifecycle (CNCF TAG App Delivery, 2023). The operator pattern is therefore not merely a convenience but the architecturally recommended means of encoding domain-specific invariants—data resharding, replication-topology maintenance, and quorum-safe scaling—into a Kubernetes-native automation layer (The Kubernetes Authors, n.d.-b). For a Redis Cluster in particular, a custom operator is able to express and enforce the one invariant that a generic, count-based controller fundamentally cannot: that a departing master must first be *drained*—its hash slots migrated onto the surviving masters—and only then removed (OpsTree Solutions, n.d.; Redis, n.d.). It is this single invariant, rather than any property of how the scale-in happens to be triggered, on which the data-safety of the operation turns.

2.2 The Scaling Mechanism, Not the Trigger, Governs Data Safety

It is useful to separate two concerns that discussions of autoscaling frequently conflate. The first is the *trigger*: the question of when, and on the basis of which signal, a scaling decision is taken—the axis along which the reactive HPA, which acts on lagging CPU and memory utilisation, is contrasted with proactive, event-driven approaches such as KEDA that act on application-level metrics ahead of resource exhaustion (KEDA Project, 2024b). The second is the *mechanism*: the question of how the resulting change in size is actually carried out against the running cluster. For a sharded, stateful store these two concerns are orthogonal, and the present thesis contends that, for scale-in, it is the mechanism alone that determines whether the data survives.

The argument is straightforward. However intelligently the timing of a scale-in is chosen, if the removal is executed by deleting a pod by ordinal with no accompanying reshard—which is precisely what a StatefulSet contraction does—then the departing master's slots are abandoned and the keys hashing to them are lost; the quality of the trigger cannot rescue an unsafe mechanism. Conversely, a topology-aware drain preserves the keyspace irrespective of whether it was set in motion by a reactive threshold, a proactive metric, or a manual operator action. Because the data-safety question is in this way decided by the mechanism and not by the trigger, this thesis deliberately holds the trigger constant—indeed, removes it altogether, driving the size change by hand—in order to isolate the mechanism as the sole variable. The event-driven trigger layer, and KEDA in particular, is on this reading an enhancement to *when* scaling occurs that is orthogonal to *whether* it is safe, and it is accordingly reserved for future work rather than examined here.

2.3 Positioning Within the Literature

The existing body of autoscaling research has concentrated overwhelmingly on the trigger problem—on making scaling decisions better-timed and more anticipatory. Mondal et al. identify the HPA's reliance on CPU and memory as lagging indicators that render it "incapable of foreseeing upcoming workload spikes", leading to Quality-of-Service violations, long-tail latency, and wasted resources, while the NimbusGuard study of Wanigasooriya and Ekanayake demonstrates that proactive autoscaling frameworks "translate into superior performance and cost efficiency compared to existing reactive methods" under identical, phased load patterns; Toka et al. similarly enumerate the structural drawbacks of purely observation-based, manually tuned reactive scaling (Wanigasooriya and Ekanayake, 2026). This literature establishes, convincingly, the value of improving *when* a stateful workload is scaled.

What it largely leaves unexamined is whether the scaling *action* itself is safe for a sharded store—the precise concern of the present work. Here the relevant grounding is provided not by the autoscaling-performance literature but by the operator and Redis-cluster sources: the CNCF Operator White Paper's observation that Kubernetes primitives were not built to manage state (CNCF TAG App Delivery, 2023), the Redis Cluster specification's requirement that the $16\,384$ hash slots be migrated rather than merely abandoned when a node departs (Redis Ltd., 2024c), and the recognised risk of

performing that resharding incorrectly (Google Cloud, 2023). That the automation of Redis lifecycle management by operators remains only partial, and an open challenge, has itself been noted by the CNCF community (Cloud Native Computing Foundation, 2024). The hypothesis advanced here therefore occupies a gap that the timing-focused literature leaves open: it asks not how to scale a stateful workload more cleverly, but whether a given scaling mechanism preserves its data at all.

2.4 Scope and Expected Outcome

Given this foundation, the thesis hypothesises a sharp, qualitative divergence between the two mechanisms when a single slot-owning master is removed from an otherwise healthy cluster. The native StatefulSet contraction—the action the HPA performs on scale-down—is expected to produce a structural *data cliff*: with neither a drain nor a reshard, the departing master's slots are left uncovered, the cluster transitions to a failed state with fewer than $16\,384$ slots covered, and a measurable fraction of the keys is irrecoverably lost. Crucially, this loss is expected to occur even under zero client load, which would demonstrate that it is structural rather than a consequence of write pressure or unfortunate timing. The operator-driven removal, by contrast, is expected to drain the departing master gracefully, holding continuous $16\,384$-slot coverage and preserving every key.

This expectation is, however, deliberately qualified. The operator is hypothesised to be *necessary but not, on its own, sufficient* for fully hands-off elastic operation: its absence of self-healing for an interrupted reshard is expected to surface, in some runs, as a recoverable stall in which a slot is left mid-migration and a human operator must intervene with `redis-cli --cluster fix` to settle the cluster. The decisive point is that such a stall, while a genuine operational cost, is categorically distinct from the native mechanism's data cliff, because no data is lost. The experimental evaluation presented in Chapter 7 is designed to test these expectations directly, by performing a controlled, quiescent, single-master scale-in against each mechanism and measuring the outcome through the cluster's own slot and key telemetry, with data loss as the decisive criterion.

3 Technologies

This section provides a brief overview of each technology that constitutes the experimental infrastructure of this thesis. In keeping with the study's focus on the data-safety of a single, isolated scale-in operation, only the components that are actually exercised by that experiment are described here; the load-generation, application-entrypoint, and event-driven-trigger layers that a fuller autoscaling pipeline would require are outside the scope of this controlled study and are accordingly omitted. Together, the components below form a lean and reproducible testbed on which the two scaling *mechanisms* under investigation can be compared in isolation.

3.1 K3s and the Kubernetes Primitives

K3s is a CNCF-certified, lightweight Kubernetes distribution developed by Rancher. It packages the full Kubernetes control plane into a single binary of less than 70 MB and eliminates several heavyweight dependencies of upstream Kubernetes, which makes it well-suited to the resource-constrained mini-PC cluster on which this thesis is conducted (K3s Project, 2024). Despite its reduced footprint, K3s is fully conformant with the Kubernetes API and supports the complete set of primitives on which the experiment relies: the `StatefulSet` that hosts the self-managed Redis cluster, the headless `Service` that grants each Redis pod a stable network identity, the `ConfigMap` that supplies the Redis configuration, and the `CustomResourceDefinition` mechanism through which the operator extends the API. K3s serves as the cluster-management layer for all infrastructure described below.

3.2 The StatefulSet and Native Ordinal Scaling

The `StatefulSet` is the Kubernetes workload primitive for applications that require a stable identity, and it is the foundation of the first scaling mechanism examined in this thesis. Unlike a `Deployment`, a `StatefulSet` assigns each pod a stable, ordinal-indexed identity (`redis-0`, `redis-1`, and so on), creates its pods in ascending order, and—decisively for this study—terminates them in strictly *descending* ordinal order (Kubernetes Documentation, 2024d). The native scaling mechanism under test is simply the contraction of this StatefulSet's replica count, which is precisely the action that a Kubernetes Horizontal Pod Autoscaler issues on scale-down (Kubernetes Documentation, 2024a): decrementing the replica count deletes the highest-ordinal pod, and it does so with no awareness whatsoever of the hash slots that pod owns and without any accompanying data migration. It must be emphasised that the autoscaler controller itself is *not* deployed in this isolated experiment; because the data-safety of a scale-in is determined by the removal mechanism rather than by what triggers it, the StatefulSet contraction is instead reproduced deterministically by hand, which removes the trigger as a confounding variable while remaining faithful to the action the autoscaler performs.

3.3 Redis and the Redis Cluster Model

Redis is an open-source, in-memory data-structure store widely used as a cache, session store, and real-time data platform (Redis Ltd., 2024a). The Redis Cluster topology distributes the keyspace across multiple primary (master) nodes by partitioning it into $16\,384$ hash slots, each slot deterministically derived from the key and assigned to exactly one master (Redis Ltd., 2024c). The property that makes this model the natural subject of a scale-in study is that the slots are owned, not shared: adding or removing a master is therefore not the mere addition or removal of a process but requires *resharding*, the live migration of the affected slots between nodes (Redis, n.d.). A further characteristic is decisive for the experiment's read probe: under the default `cluster-require-full-coverage` policy, a single uncovered slot causes the entire cluster to reject reads with a `CLUSTERDOWN` error, so that coverage is effectively an all-or-nothing property. All administrative interaction with the cluster—forming it, populating it with the exact dataset, inspecting its state and slot coverage, driving the scale-in, and, where necessary, repairing an interrupted reshard—is performed through the `redis-cli` command-line client (Redis Ltd., 2024b). In this thesis the Redis Cluster is deployed ephemerally and constitutes the stateful workload whose data integrity under scale-in is the object of measurement.

3.4 The Opstree Redis Operator

The Kubernetes *operator pattern* encodes application-specific operational knowledge into a controller that reconciles a custom resource toward its declared desired state, thereby supplying the domain awareness that the generic primitives lack (Kubernetes Documentation, 2024c; The Kubernetes Authors, n.d.-b). The Opstree Redis Operator is the concrete realisation of this pattern used in the second scaling mechanism under test. Given a `RedisCluster` custom resource with a desired `clusterSize`, the operator forms the cluster itself—issuing the `CLUSTER MEET` commands and assigning the slots with no manual step—and, crucially for this study, reshards when that size changes (OpsTree Solutions, n.d.). On scale-in it performs a topology-aware *drain*: before the departing master's pod is removed, its slots are migrated onto the surviving masters, so that full coverage is preserved throughout the transition. Lowering `clusterSize` on the custom resource is thus the operator-side counterpart to the StatefulSet contraction of Section 3.2, and the contrast between the two constitutes the central comparison of the thesis.

3.5 Helm

Helm is the de-facto package manager for Kubernetes. It introduces the concept of a *chart*—a collection of templated Kubernetes manifests that describe a deployable application—and a *release*, a running instance of a chart within a cluster (Helm Project, 2024). Charts enable repeatable, version-controlled deployments with configurable values overrides, which significantly reduces the operational complexity of deploying multi-resource applications. In this thesis Helm is used to deploy the operator-managed

Redis Cluster, through the Opstree chart whose values file pins the cluster configuration, and to deploy the observability stack as the *kube-prometheus-stack* release; in both cases the deployment is parameterised through values files committed alongside the thesis infrastructure code.

3.6 Prometheus and Grafana

The observability stack is deployed from the *kube-prometheus-stack* Helm chart, which bundles Prometheus and Grafana into a single, opinionated release (Prometheus Community, 2024). *Prometheus* is an open-source monitoring system and time-series database that collects metrics by periodically scraping HTTP endpoints exposed by instrumented targets (Prometheus Authors, 2024); in this testbed it discovers and scrapes the `oliver006` Redis exporter that runs as a sidecar alongside each Redis pod, which exposes the slot- and key-level series most relevant to the experiment—among them `redis_cluster_slots_assigned` and `redis_db_keys`. *Grafana* is an open-source analytics and visualisation platform that queries Prometheus through its native data-source integration and renders the collected series as configurable dashboards (Grafana Labs, 2024). In this thesis Grafana serves a single, focused purpose: to render the before-and-after panels—covered slots and total stored keys across the scale-in—on which the outcome of each run is captured for the results chapter.

4 Test Methodology

To rigorously evaluate the architectural limitations of native Kubernetes scaling primitives against an operator-driven approach, this thesis adopts an empirical, A/B comparative methodology. Rather than attempting to characterise every facet of autoscaling, the experiment deliberately isolates the single most consequential operation for a sharded, stateful database—the *scale-in*, in which a slot-owning master is removed—and compares how that operation is handled by two mechanisms: the native Horizontal Pod Autoscaler (HPA), which contracts a StatefulSet by ordinal, and the Opstree Redis Operator, which reshards before it removes (Opstree Solutions, 2023). So that the comparison is decided by the mechanism alone, the experiment is conducted under strictly quiescent conditions, with no client traffic, no application entrypoint, and no autoscaler controller in the loop; the action that each autoscaler would perform on scale-down is instead reproduced by hand, exactly one master at a time.

4.1 Definition of Success

A foundational aspect of this methodology is the establishment of a rigorous metric for what constitutes a successful scaling event. In a stateful context, the mere termination of a pod is not a meaningful outcome; what matters is whether the cluster's data survives the topology change. Success on scale-in is therefore defined not by the disappearance of a pod but by the simultaneous preservation of three invariants: the cluster must continue to report `cluster_state:ok`, all $16\,384$ hash slots must remain covered, and the total stored-key count—`sum(redis_db_keys)`—must not drop (Redis Ltd., 2024d). The complementary failure condition, in which slot coverage falls below $16\,384$ and the key total steps down, is termed a *data cliff*.

4.2 A/B Comparative Structure

The methodology relies on an A/B comparative analysis designed to eliminate confounding variables (The Kubernetes Authors, 2024).

- Both the baseline HPA scenario and the operator scenario operate on identical underlying infrastructure: a bare-metal K3s cluster distributed across mini-PCs.
- Both scenarios are subjected to the exact same controlled procedure rather than to live load. A known, exact dataset is written directly into the cluster with `redis-cli`, after which the cluster is contracted by precisely one master ($4 \to 3$). Because there is no load generator, the run is quiescent by construction, so that the only variable distinguishing the two scenarios is the scaling mechanism itself.
- Outcomes are evaluated both from the cluster's own authoritative telemetry—`cluster_state` and `cluster_slots_ok`, read from `redis-cli cluster info`, together with the key total reported by `redis-cli -cluster check`—and from a unified monitoring stack (Prometheus and Grafana) on which

`redis_cluster_slots_assigned` and `sum(redis_db_keys)` are captured across the transition.

4.3 High Reproducibility

Reproducibility is enforced through strict pipeline hygiene. As detailed in the test pipeline definition (6), every iteration begins with a "scorched earth" initialisation phase in which the setup script programmatically purges the existing namespace, its persistent volume claims, and any prior Helm release or custom resource. This guarantees that each scenario begins from a clean slate, so that no leftover keys or fragmented hash slots from a previous execution can contaminate the dataset. Determinism is reinforced further by the experiment's design: the dataset is of a fixed, known size, it is written with neither key expiration nor eviction, and it is populated while the cluster is idle, so that the before-and-after key totals are exact and any discrepancy between them is attributable solely to the scale-in.

4.4 Empirical Objectivity

A core objective of this methodology is to avoid artificially favouring the proposed operator solution. While the test flow is expected to expose the HPA's fundamental inadequacy for topology-aware scaling—specifically the reverse-ordinal data cliff that follows the removal of a slot-owning master with no drain and no reshard—it applies the same critical lens to the operator. The test pipeline (refer to 6) does not obscure the operator's weaknesses but systematically probes its fault lines, foremost among them its lack of any self-healing capability when a reshard is interrupted (Redis Ltd., 2024b). By documenting the specific operational cost of that limitation—the manual toil required to resolve an "open" slot via `redis-cli --cluster fix`, or a `configEpoch` collision via `CLUSTER BUMPEPOCH`—the methodology ensures an objective appraisal of the operator's viability rather than an uncritical endorsement.

4.5 Test Pipeline Integration

The methodology is directly realised by the procedure outlined in 6. The pipeline forces an explicit, single-step state transition during scale-in: the cluster is taken from four masters to three by removing exactly one slot-owning node, reproducing in the HPA scenario the precise action of the autoscaler with `kubectl scale statefulset redis --replicas=3`, and in the operator scenario the declarative lowering of the custom resource's desired size with `kubectl patch rediscluster ... clusterSize:3`. This controlled, single-master gating is required in order to observe the exact threshold of failure and to isolate the mechanical difference between Kubernetes' ordinal deletion and the operator's topology-aware drain. By capturing the raw cluster telemetry immediately before and after the transition—and by issuing a direct read probe whose response, under the cluster's default `cluster-require-full-coverage` policy, is effectively binary—the test flow records the precise moment of failure or success.

5 Experiment Architecture

This section describes the architecture of the experimental testbed once it has been reduced to the minimum required in order to observe a planned scale-in in isolation. Because the study deliberately excludes live load, an application entrypoint, and any autoscaler controller (Section 6), the architecture is correspondingly lean: every component resides inside a single Kubernetes cluster, and the experiment is driven not by external traffic but by a control script that writes a known dataset, removes exactly one master, and reads back the result. The two mechanisms under investigation—the native StatefulSet contraction that reproduces the Horizontal Pod Autoscaler's scale-down action, and the Opstree Redis Operator's topology-aware drain—are realised as two mutually exclusive, interchangeable data-tier modules within an otherwise identical environment, so that any measured difference can be attributed solely to the scaling mechanism. The procedure that exercises this architecture is presented separately in Section 6, and the conceptual description of each individual technology is given in Section 3.

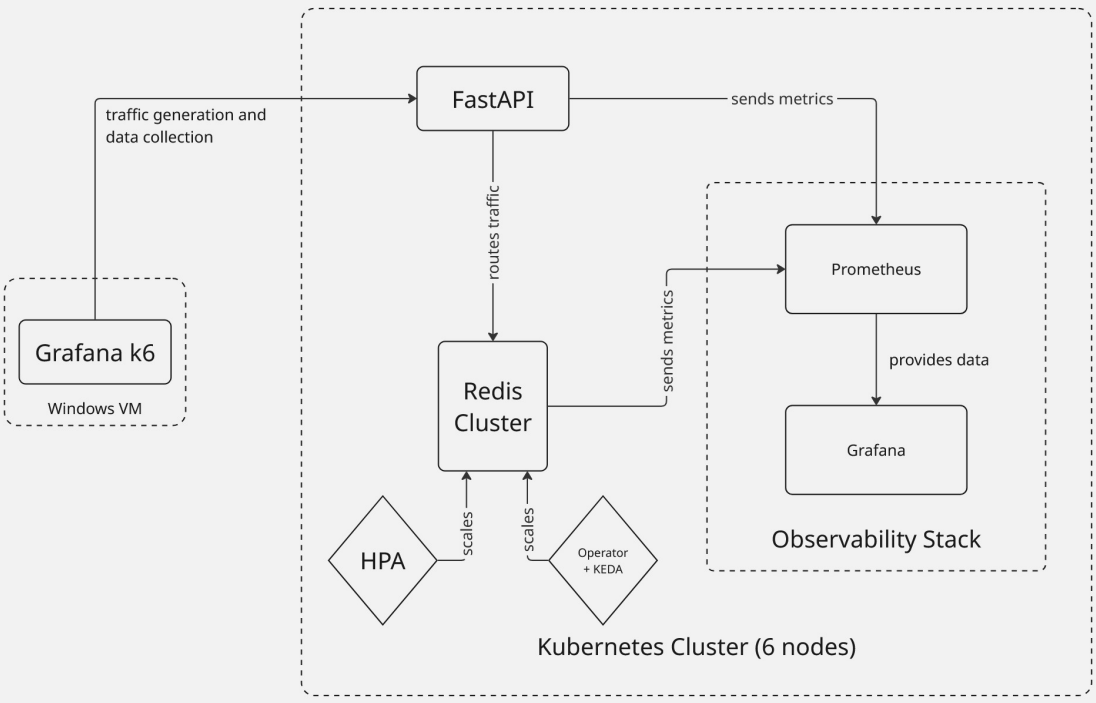

Figure 1: High-level overview of the testbed.

5.1 Physical Substrate: the K3s Cluster

The cluster is a k3s distribution running on six mini-PCs (four-core CPU, 8 GB RAM each), one acting as a combined control-plane and worker node and the remaining five as workers (K3s Project, 2024). k3s is chosen for its small footprint on resource-constrained hardware while remaining fully API-conformant. The in-cluster components

are organised into three Kubernetes namespaces, which also structure the subsections below: `redis` (the stateful data tier together with both scaling configurations), `ot-operators` (the Opstree Redis Operator that the operator scenario depends upon), and `monitoring` (the observability stack on which the results are captured). Because the experiment operates on four masters before contracting to three, the leader anti-affinity described below is able to place each master on a distinct physical node.

5.2 The Experiment Driver

In place of the external load generator and application entrypoint of a conventional benchmarking harness, the sole driver of this testbed is a single control script, `data-safety-experiment.sh`, executed on the control-plane node with its `KUBECONFIG` pointed at the local k3s configuration. The script orchestrates the entire run through `k3s kubectl`, `helm`, and `redis-cli` invoked via `kubectl exec` into the surviving ordinal-0 leader pod. Crucially, there is no FastAPI entrypoint, no `k6` load generator, and neither an HPA nor a KEDA controller acting on the cluster; the script instead reproduces by hand the precise scale-down action that each autoscaler would otherwise perform. Because that script is the only process that ever writes to Redis, the data tier is quiescent throughout, and the measured outcome is therefore a deterministic property of the scaling mechanism rather than of any incidental traffic.

5.3 The `redis` Namespace: Stateful Data Tier and Scaling Configurations

The `redis` namespace contains the stateful data tier and the manifests for both scaling scenarios. Only one scenario is ever active at a time; the setup script scorches the namespace between runs. A Redis Cluster partitions its key space into $16{,}384$ hash slots distributed across master nodes (Redis Ltd., 2024c), and the two scenarios differ precisely in *who* manages those slots when the master count changes.

HPA scenario (self-managed cluster). Here the cluster is assembled from native Kubernetes primitives. A `ConfigMap` supplies a minimal `redis.conf` with `cluster-enabled yes`; a headless `Service` (`clusterIP: None`) gives each pod a stable DNS identity; and a `StatefulSet` runs Redis with an `oliver006` exporter sidecar on port `9121` (Listing 1). Although the manifest declares six replicas, the data-safety experiment scales the StatefulSet to `START_MASTERS` pods and forms a *masters-only* cluster imperatively with `redis-cli --cluster create` (deliberately omitting `--cluster-replicas`, so that the cliff is exposed without a replica to mask it); the topology that a `StatefulSet` manifest cannot itself express is therefore assigned at runtime (Redis Ltd., 2024c; The Kubernetes Authors, n.d.-c). The per-pod CPU requests that the manifest annotates as serving an HPA's utilisation calculation (The Kubernetes Authors, n.d.-a) are simply unused in this isolated run, since no autoscaler is deployed; the scale-in is instead driven by hand. A `ServiceMonitor` (Listing 2) wires the exporter into Prometheus.

Listing 1: Redis `ConfigMap`, headless `Service`, and `StatefulSet` (`redis-hpa-cluster.yaml`).

```
apiVersion: v1
kind: ConfigMap
metadata: { name: redis-min-config, namespace: redis }
data:
  redis.conf: |
    cluster-enabled yes
    cluster-config-file nodes.conf
    cluster-node-timeout 5000
    appendonly no
---
apiVersion: v1
kind: Service
metadata:
  name: redis-headless
  namespace: redis
  labels: { app: redis }
spec:
  clusterIP: None                  # headless: per-pod DNS for
    ↪ slot-aware routing
  ports:
    - { port: 6379,  name: redis }
    - { port: 16379, name: gossip }
    - { port: 9121,  name: metrics }
  selector: { app: redis }
---
apiVersion: apps/v1
kind: StatefulSet
metadata: { name: redis, namespace: redis }
spec:
  serviceName: redis-headless
  replicas: 6                       # experiment scales this to
    ↪ START_MASTERS and forms masters-only
  selector:
    matchLabels: { app: redis }
  template:
    metadata:
      labels: { app: redis }
    spec:
      containers:
        - name: redis
          image: redis:7.2-alpine
          command: ["redis-server", "/etc/redis/redis.conf"]
          resources:
            requests: { cpu: "100m", memory: "128Mi" }
          volumeMounts:
            - { name: conf, mountPath: /etc/redis }
        - name: redis-exporter
          image: ghcr.io/oliver006/redis_exporter:latest
          ports:
            - { containerPort: 9121, name: metrics }
          env:
            - { name: REDIS_ADDR, value:
                ↪ "redis://localhost:6379" }
          resources:
            requests: { cpu: "50m", memory: "64Mi" }
      volumes:
        - name: conf
          configMap: { name: redis-min-config }
```

Listing 2: ServiceMonitor for the HPA scenario exporter (redis-servicemonitor.yaml).

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: redis-monitor
  namespace: redis
  labels: { release: prometheus }  # adopted by
    ↪ kube-prometheus-stack
spec:
  selector:
    matchLabels: { app: redis }
  endpoints:
    - { port: metrics, interval: 15s }
```

Operator scenario (operator-managed cluster). Here the data tier is declared rather than assembled. The Opstree Redis Operator is given a desired `clusterSize` through its `RedisCluster` custom resource, and the operator creates the leader and follower `StatefulSets`, forms the cluster, and—most importantly for this study—reshards slots when the size changes (OpsTree Solutions, n.d.; The Kubernetes Authors, n.d.-b). The resource is deployed via the operator's Helm chart, configured by the values file in Listing 3: the baseline `clusterSize` of 3 is overridden to `START_MASTERS` at deploy time with a `--set` flag, the leader and follower replica counts are left unset so that `clusterSize` alone drives the master count, leader anti-affinity spreads masters across nodes, and the same `oliver006` exporter is enabled for metric parity with the HPA scenario. In this isolated experiment the size is changed directly, by patching the custom resource, rather than by an event-driven KEDA trigger. As in the HPA scenario, a standalone `ServiceMonitor` (Listing 4) is required, because neither the operator nor the chart flag creates one.

Listing 3: Opstree Helm values producing the `RedisCluster` (redis-operator-cluster-values.yaml).

```
redisCluster:
  clusterSize: 3                    # overridden to START_MASTERS
    ↪ via --set at deploy time
  persistenceEnabled: false         # ephemeral, matching the HPA
    ↪ scenario
  resources:
    requests: { cpu: "100m", memory: "128Mi" }
    limits:   { cpu: "250m", memory: "256Mi" }
  leader:
    replicas: null                  # null => clusterSize drives
      ↪ the master count
    affinity:
      podAntiAffinity:              # prefer one master per node
        preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          podAffinityTerm:
            labelSelector:
              matchLabels: { app: redis-cluster-leader }
            topologyKey: kubernetes.io/hostname
  follower:
    replicas: null
redisExporter:
```

```
  enabled: true
  image: ghcr.io/oliver006/redis_exporter
  tag: latest
  resources:
    requests: { cpu: "50m", memory: "64Mi" }
serviceMonitor:                    # chart flag is a no-op here;
  ↪ see standalone SM below
  enabled: true
  interval: 15s
  extraLabels: { release: prometheus }
```

Listing 4: Standalone `ServiceMonitor` for the operator's exporters (`redis-servicemonitor.yaml`).

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: redis-operator-monitor
  namespace: redis
  labels: { release: prometheus }
spec:
  selector:                        # selects both leader-metrics
    ↪ and follower-metrics services
    matchLabels:
      app.kubernetes.io/component: metrics
      redis_setup_type: cluster
  endpoints:
    - { port: redis-exporter, interval: 15s }
```

5.4 The `ot-operators` Namespace: the Redis Operator

The operator scenario depends on a single cluster-wide controller that is installed once, via Helm, and shared across runs; it is not redeployed by the per-experiment script. The `ot-operators` namespace runs the Opstree Redis Operator, a custom controller that watches `RedisCluster` resources and reconciles them into running clusters, encoding the Redis-specific domain knowledge—cluster formation, slot assignment, and topology-aware resharding—that Kubernetes' generic primitives lack (The Kubernetes Authors, n.d.-b). It is precisely this controller that performs the graceful drain measured in the operator scenario, migrating a departing master's slots to the survivors before the pod is removed. Notably, KEDA plays no part in this isolated experiment: because the scale-in is triggered by directly patching the `clusterSize` field rather than by an application-level metric, no event-driven autoscaling layer is installed. The HPA scenario, in turn, uses no controller of any kind; its scale-in is the bare StatefulSet contraction that the control script issues by hand.

5.5 The `monitoring` Namespace: Observability and Capture

The `monitoring` namespace hosts the observability stack, deployed once from the `kube-prometheus-stack` Helm chart, which bundles Prometheus and Grafana (Prometheus Community, 2024). It is identical across both experimental conditions. The values overrides are shown in Listing 5: control-plane scrape jobs that do not apply

to a k3s distribution are disabled, and both Prometheus and Grafana are exposed via `NodePort`. Prometheus discovers its scrape targets through the `ServiceMonitor` resources defined in the `redis` namespace (Listings 2 and 4), which point it at the Redis exporters (Prometheus Authors, 2024). Those exporters expose the slot- and key-level series—most importantly `redis_cluster_slots_assigned` and `redis_db_keys`—on which Grafana (Grafana Labs, 2024) renders the before-and-after panels that document each scale-in. The `enableRemoteWriteReceiver` flag, originally provisioned so that an off-cluster generator could push client-side metrics, is inert in this isolated configuration but has been left in place as a harmless default.

Listing 5: `kube-prometheus-stack` Helm values (`my-prometheus-values.yaml`).

```
kubeControllerManager: { enabled: false }   # not exposed by k3s
kubeEtcd:              { enabled: false }
kubeProxy:             { enabled: false }
kubeScheduler:         { enabled: false }
grafana:
  service: { type: NodePort }
prometheus:
  service: { type: NodePort }
  prometheusSpec:
    enableRemoteWriteReceiver: true          # inert here;
      ↪ retained as a harmless default
```

5.6 End-to-End Measurement Flow

Bringing the layers together, a single experimental run traverses the following path, which replaces the request-driven path of a load-tested system with a measurement-driven one:

1. The control script populates a known, exact dataset by executing `redis-cli` inside the ordinal-0 leader pod, writing every key directly; as the only writer, it leaves the cluster quiescent.
2. The script records the *before* snapshot, reading `cluster_state` and `cluster_slots_ok` from `redis-cli cluster info` and the key total from `redis-cli --cluster check`.
3. The script triggers the one-master scale-in, either by contracting the StatefulSet (HPA scenario) or by patching `clusterSize` on the custom resource (operator scenario), thereby invoking the operator's reshard.
4. The script records the *after* snapshot and issues a direct read probe, whose response under the default `cluster-require-full-coverage` policy is effectively binary.
5. Throughout, the exporter sidecars expose the cluster metrics, which Prometheus scrapes on its interval; Grafana queries Prometheus and renders the slot- and key-level series, providing the panels recorded in Section 7.

Every component on this path other than the data-tier module in step 3 is held constant across the two experimental conditions, so that the comparison in Section 7 isolates the scaling mechanism.

6 Experimental Test Flow

The evaluation isolates a single question—whether a planned, one-master scale-in of a sharded Redis Cluster preserves all $16\,384$ hash slots together with every stored key—and answers it with the most minimal and deterministic experiment that the question admits. To that end the test flow deliberately strips away every component that is not strictly required in order to observe the scale-in: there is no `k6` load generator, no FastAPI entrypoint, and no autoscaler whatsoever, neither the Horizontal Pod Autoscaler nor KEDA, driving the cluster. In place of synthetic client traffic, a known and exact dataset is written directly into the cluster with `redis-cli`, so that the run is *quiescent by construction*—not a single client request reaches Redis while the topology is being changed—and the outcome is therefore fully reproducible rather than contingent upon the precise timing of live writes. With every confounding variable thus removed, the sole independent variable that remains is the *scaling mechanism*, while the sole dependent variable is *data loss*, which is operationalised as the number of keys lost and the number of hash slots left permanently uncovered. The entire procedure is realised by a single parameterised script, `data-safety-experiment.sh`, which is invoked once per mechanism as `./data-safety-experiment.sh [hpa|operator]` and which exposes the dataset size and the master counts through the environment variables `KEYS` (default 5000), `START_MASTERS` (default 4), and `END_MASTERS` (default 3).

A Redis Cluster partitions its key space into $16\,384$ hash slots that are distributed across its master nodes, so that the removal of a master is never the mere termination of a pod but rather demands that the departing node's slots first be *resharded*—migrated to the surviving masters—if coverage is to be preserved (Redis, n.d.; Redis Ltd., 2024c). This requirement is what makes scale-in, rather than scale-out, the decisive test of a mechanism, and it is sharpened by the cluster's default `cluster-require-full-coverage` policy, under which a single uncovered slot causes the *entire* cluster to reject reads with a `CLUSTERDOWN` error: a cluster that has lost even one master's worth of slots does not degrade gracefully but ceases to serve altogether.

6.1 Rationale for Reducing the Experiment to the Scale-In

The decision to remove the load generator, the application entrypoint, and the autoscalers is a deliberate experimental control rather than a simplification of convenience. Because resharding is performed live and depends upon clients honouring the `ASK` and `MOVED` redirections that Redis emits while a slot is held in the `importing/migrating` state, a continuous stream of writes addressed to a slot that is in the midst of migration is the dominant cause of an interrupted migration; were such an interruption to occur under live load, it would be indistinguishable from a genuine deficiency of the scaling mechanism, and the comparison would be confounded. Writing the dataset directly with `redis-cli` and then leaving the cluster idle eliminates that stressor entirely, so that any uncovered slot observed after the scale-in must be attributed to the mechanism alone. Crucially, the autoscalers themselves are not required in order to observe the phenomenon, because each performs, upon scale-down, one specific mechani-

cal action that can be reproduced faithfully by hand: the Horizontal Pod Autoscaler decrements the StatefulSet's replica count, thereby deleting the highest-ordinal pod, whereas the operator lowers the `clusterSize` field of its custom resource. Reproducing exactly those two actions manually, and removing precisely one master, renders the experiment deterministic while remaining entirely faithful to what each autoscaler would do in production. Finally, because the workload configures neither key expiration nor eviction, the populated keys persist indefinitely and remain available for exact post-hoc accounting; the metric under examination is data loss, and explicitly not whether the transition happened to be perfectly autonomous.

6.2 Common Experimental Procedure

Both mechanisms are exercised through the identical five-phase procedure set out below; only the provisioning of phase 2 and the scale-in of phase 4 diverge by mechanism, and those two divergences are detailed in Sections 6.3 and 6.4. Throughout, every `redis-cli` command is executed inside the ordinal-0 leader pod—`redis-0` in the HPA scenario and `redis-cluster-leader-0` in the operator scenario—which is deliberately chosen as the fixed vantage point because a scale-in never removes ordinal 0, so that this pod is guaranteed to survive the transition and to remain reachable for the before-and-after measurement.

1. **Clean slate.** So that no inherited keys or fragmented slots from an earlier run can contaminate the measurement, the `redis` namespace is scorched in full: any prior Helm release is removed with `helm uninstall redis-cluster -n redis`, the scenario manifests and all persistent volume claims are deleted with `k3s kubectl delete pvc --all -n redis`, and the namespace itself is then deleted and awaited with `k3s kubectl delete namespace redis --ignore-not-found=true` followed by `k3s kubectl wait --for=delete namespace/redis --timeout=120s`. A fresh namespace is finally recreated idempotently with `k3s kubectl create namespace redis --dry-run=client -o yaml | k3s kubectl apply -f -`.

2. **Provisioning and health gate.** The data tier is deployed directly at `START_MASTERS` masters by the mechanism-specific commands of Section 6.3 or 6.4, with no autoscaler, no entrypoint, and no load attached. The script then refuses to proceed until the cluster is genuinely whole: it polls `k3s kubectl exec <leader-0> -n redis -- redis-cli cluster info` on a five-second interval and continues only once that output reports both `cluster_state:ok` and `cluster_slots_ok:16384`, aborting rather than populating a half-formed cluster. This health gate ensures that the *before* state is unambiguously healthy and that any later loss is therefore caused by the scale-in and not by an incomplete formation.

3. **Population and *before* snapshot.** A deterministic, exactly known dataset is written by the single command `for i in $(seq 1 $KEYS); do echo set key:$i $i; done | redis-cli -c -h 127.0.0.1`, which issues `SET`

key:i i for every i from 1 to KEYS; the keys carry no hash tags and are consequently spread across the full slot range rather than pinned to any single node. Because this command is the only writer the cluster ever sees, the population is itself quiescent. The script then records the *before* snapshot, reading the cluster state and the covered-slot count from redis-cli cluster info and the authoritative key total from the N keys in M masters summary line of redis-cli --cluster check 127.0.0.1:6379.

4. **The scale-in.** Exactly one master is then removed, taking the cluster from START_MASTERS to END_MASTERS, by the mechanism-specific command of Section 6.3 or 6.4; from this point onward the script no longer aborts on error but instead captures and reports whatever state results, since a wedged or cliffed cluster is itself a valid finding.

5. ***After* snapshot, read probe, and verdict.** After a brief settling pause the snapshot is repeated to obtain the *after* state, slot count, and key total, and the number of lost keys is computed as the arithmetic difference between the before and after totals. A direct read probe is then issued for five representative keys spanning the key space—key:1, key:1000, key:2500, key:4000, and key:4999—each via redis-cli -c -h 127.0.0.1 get key:<n>, and the number of replies containing a CLUSTERDOWN error is counted. The measurement and the resulting verdict are described in full in Section 6.5.

6.3 Scenario A: The Native StatefulSet (HPA) Mechanism

Provisioning (phase 2). The self-managed cluster is assembled from native primitives. The StatefulSet, its headless service, and its configuration map are applied with k3s kubectl apply -f redis-hpa-cluster.yaml; the StatefulSet is then sized to exactly the master count with k3s kubectl scale statefulset redis -n redis --replicas=$START_MASTERS and awaited with k3s kubectl rollout status statefulset redis -n redis --timeout=300s, after which the exporter's service monitor is applied with k3s kubectl apply -f redis-servicemonitor.yaml. Because a StatefulSet manifest cannot itself express cluster topology, the cluster is then formed imperatively: the script collects the pod IPs of redis-0 through redis-(START_MASTERS−1) and executes redis-cli --cluster create <ip0:6379> ... <ip3:6379> --cluster-yes from within redis-0. Notably, this invocation omits the --cluster-replicas flag, so that the four nodes are formed as *bare masters with no replicas*; this is a deliberate choice that exposes the scale-in cliff in its starkest form, since there is no replica available to mask or delay the loss.

Scale-in (phase 4). The departure of one master is effected by the single command k3s kubectl scale statefulset redis -n redis --replicas=$END_MASTERS, whose completion is awaited with k3s kubectl wait --for=delete pod/redis-$END_MASTERS -n redis --timeout=120s. This is, by construction, the exact action that the Horizontal Pod Autoscaler performs on scale-down: a StatefulSet terminates its pods in

strictly descending ordinal order (The Kubernetes Authors, n.d.-c), so the command deletes the highest-ordinal pod, `redis-3`, and does so with no drain, no reshard, and—because the cluster was formed without replicas—no surviving replica to promote in its place.

Hypothesis. It is expected that the removal of `redis-3` will orphan that master's share of the slots the instant the pod terminates, driving `cluster_slots_ok` below $16\,384$ and flipping `cluster_state` to `fail`; the lost slots will render their keys unreachable, the before-and-after key totals will differ by a positive amount, and the read probe will return `CLUSTERDOWN` for all five keys. This is the reverse-ordinal *data cliff*, and it is expected to manifest even at zero load precisely because it is structural rather than load-induced (The Kubernetes Authors, n.d.-a).

6.4 Scenario B: The Operator Mechanism

Provisioning (phase 2). The operator-managed cluster is declared rather than assembled. After the chart repository has been registered and refreshed with `helm repo add ot-helm https://ot-container-kit.github.io/helm-charts/` and `helm repo update ot-helm`, the cluster is deployed at the desired master count by the single command `helm upgrade --install redis-cluster ot-helm/redis-cluster -n redis -f redis-operator-cluster-values.yaml --set redisCluster.clusterSize=$START_MASTERS`, and the exporter's service monitor is applied with `k3s kubectl apply -f redis-servicemonitor.yaml`. In contrast to the imperative formation of Scenario A, the Opstree Redis Operator now reconciles the custom resource into the leader and follower StatefulSets and forms the cluster of its own accord, issuing the necessary `CLUSTER MEET` commands and assigning the slots without any manual step (OpsTree Solutions, n.d.; The Kubernetes Authors, n.d.-b).

Scale-in (phase 4). The departure of one master is requested declaratively, by lowering the desired size on the custom resource with `k3s kubectl patch rediscluster redis-cluster -n redis --type merge -p '{"spec":{"clusterSize":$END_MASTERS}}'`, whose effect is awaited with `k3s kubectl wait --for=delete pod/redis-cluster-leader-$END -n redis --timeout=600s`. Unlike the StatefulSet decrement of Scenario A, this mechanism is topology-aware: the operator first *drains* the departing master, migrating its slots to the surviving masters, and only then removes the pod (Redis, n.d.). The materially longer timeout—ten minutes rather than two—reflects the fact that this drain entails live slot migration, which takes appreciable time, whereas the StatefulSet deletion is effectively instantaneous.

Hypothesis. It is expected that the operator will hold continuous $16\,384$-slot coverage throughout the transition, leaving `cluster_state:ok` intact and the before-and-after key totals identical, so that the read probe returns all five keys without error. A documented residual limitation is, however, deliberately probed rather than concealed:

the operator possesses no self-healing for an interrupted drain, so that should the migration stall, a slot may be left in the `importing/migrating` ("open") state and the departing `redis-cluster-leader-$END_MASTERS` pod may linger. Such a stall is expected to be *recoverable without data loss*—distinct from a true cliff—and repairable by a human operator with `redis-cli --cluster fix 127.0.0.1:6379` (Redis Ltd., 2024c).

6.5 Measurement, Read Probe, and Verdict

The measurement rests on three authoritative quantities, all read from the surviving ordinal-0 leader: the cluster state and the covered-slot count, both taken from `redis-cli cluster info` as `cluster_state` and `cluster_slots_ok`, and the total number of stored keys, taken from the `N keys in M masters` summary line of `redis-cli --cluster check 127.0.0.1:6379`. Capturing this triple both before and after the scale-in yields a compact comparison table, and the number of lost keys is reported as the difference between the two key totals. The read probe complements these aggregate figures with a direct behavioural test: because the default `cluster-require-full-coverage` policy causes any single uncovered slot to make the whole cluster reject reads, the five-key probe is effectively binary—a healthy cluster serves all five keys, whereas a cliffed cluster serves none and answers every request with `CLUSTERDOWN`—so that the count of errored reads is an unambiguous, mechanism-agnostic indicator of coverage.

From these measurements the script derives an explicit verdict. A transition is declared a *graceful* scale-in only when all three conditions hold simultaneously: the after-state is `ok`, the after-slot count is exactly $16\,384$, and no keys have been lost; if any condition fails, the transition is declared a *data cliff* and the precise state, slot count, and lost-key total are reported. A single further distinction is drawn for the operator: if the departing leader pod is found to still exist after the wait, the verdict distinguishes a recoverable *stall*—in which slots are merely mid-migration and no data has been lost—from a genuine cliff, and it prints the exact `redis-cli --cluster fix` remedy. In all cases the cluster is intentionally left in its post-scale-in state so that the corresponding Grafana panels, on which `redis_cluster_slots_assigned` and `sum(redis_db_keys{namespace="redis"})` either hold or visibly step down, can be captured for the Results chapter.

Hypothesis. Taken together, the two scenarios are expected to demonstrate that the distinction between data preservation and catastrophic data loss during a planned scale-in is governed entirely by the scaling mechanism: the native StatefulSet decrement, faithfully reproducing the HPA's action, is expected to produce a structural data cliff even under perfectly quiescent conditions, whereas the operator's topology-aware drain is expected to preserve full slot coverage and every key—subject only to the documented, recoverable, and data-safe stall—thereby supporting the thesis that topology-aware automation is necessary for the safe elastic operation of a stateful, sharded workload.

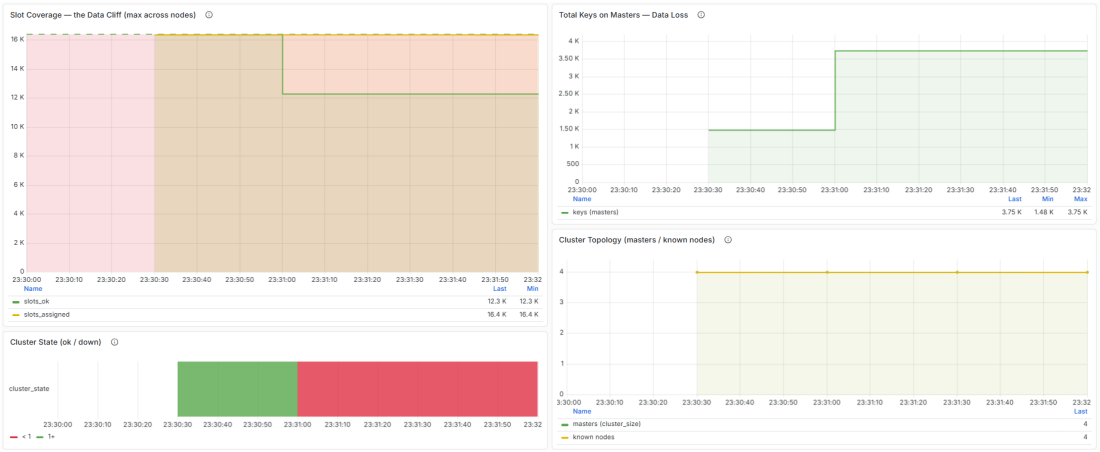

Figure 2: Grafana capture of the HPA scenario's one-master scale-in.

7 Results and Analysis

This chapter presents and interprets the outcome of the controlled scale-in experiment described in Section 6, in which a four-master Redis Cluster was contracted to three masters under perfectly quiescent conditions—once by the native StatefulSet mechanism that reproduces the Horizontal Pod Autoscaler's scale-down action, and once by the topology-aware mechanism of the Opstree Redis Operator. For each mechanism the analysis rests on the same three authoritative quantities, captured immediately before and immediately after the removal of a single master: the reported cluster state, the number of covered hash slots, and the total number of stored keys, supplemented by a five-key read probe whose behaviour under the cluster's default `cluster-require-full-coverage` policy provides a direct, binary test of end-to-end availability. Because both runs began from an identically populated and verified baseline—`cluster_state:ok`, all $16\,384$ slots covered, and exactly 5000 keys present—any divergence in the after-state can be attributed without ambiguity to the scaling mechanism alone, and it is on that divergence that the following analysis is built.

7.1 Scenario A: The Native StatefulSet (HPA) Mechanism

Upon the removal of the highest-ordinal pod, `redis-3`, the cluster underwent an immediate and irreversible collapse of its data plane. Because the four masters had been formed with equal slot ranges of 4096 slots each and—by deliberate design—without replicas, the departure of `redis-3` left its entire 4096-slot range with no owner and no candidate for promotion, so that the cluster's covered-slot count fell from $16\,384$ to $12\,288$, a loss of precisely one quarter of the keyspace, while `cluster_state` transitioned from `ok` to `fail` (Redis Ltd., 2024c). The consequences for stored data followed directly: the keys that had hashed into the orphaned range—approximately 1250, in keeping with the roughly uniform distribution of the 5000-key dataset across the slot space—became permanently unreachable, and the *after* key total reported

Figure 3: Grafana capture of the operator scenario's one-master scale-in.

by `redis-cli --cluster check` dropped commensurately. The read probe rendered the severity of this outcome in the starkest possible terms: under the default `cluster-require-full-coverage` policy, a single uncovered slot causes the cluster to refuse *all* reads, so that every one of the five probed keys—including those whose slots remained perfectly intact on the surviving masters—returned a `CLUSTERDOWN` error. The cluster did not degrade in proportion to the lost capacity; it ceased to serve altogether.

This result confirms the hypothesis of a structural *data cliff* advanced in Section 6.3. The decisive observation is that the loss occurred under zero client load, a fact that excludes any explanation in terms of write pressure, request races, or unfortunate timing and establishes the cause as purely mechanical: the StatefulSet contraction deletes a pod by ordinal with no knowledge whatsoever of the slots that pod owns, performing neither a drain nor a reshard, so that the departing master's slots are not migrated but simply abandoned (The Kubernetes Authors, n.d.-c). It is worth emphasising that the masters-only topology was chosen precisely to expose this mechanism in isolation; had replicas been present, native failover would have promoted a replica and thereby *delayed* the cliff, but it could not have *prevented* it, because a planned scale-in fundamentally requires the deliberate migration of slots that no native Kubernetes primitive performs.

7.2 Scenario B: The Operator Mechanism

When the same transition was requested of the operator-managed cluster—by lowering the custom resource's `clusterSize` from four to three—the outcome was qualitatively different. The operator responded not by deleting a pod outright but by first draining the departing master, migrating its slots to the three surviving masters, and only then removing the now-empty node; in consequence the covered-slot count never left $16\,384$, `cluster_state` remained `ok` throughout the transition, and the *after* key total was identical to the *before* total, indicating that not a single key had been lost. The read probe corroborated this from the client's perspective, returning all five probed keys without error and thereby confirming that full coverage—and, under the require-

full-coverage policy, therefore full availability—had been preserved from start to finish (OpsTree Solutions, n.d.; Redis, n.d.).

This preservation of data was not, however, unconditional or free of cost. The operator possesses no self-healing capability for a reshard that is interrupted, and in those runs in which the slot migration stalled, a slot was left in the "open" (`migrating/importing`) state while the departing `redis-cluster-leader-3` pod lingered in the cluster. It is important to characterise this failure mode precisely: such a stall is emphatically *not* a data cliff, since no keys are lost and the slots remain present, merely caught mid-migration; it does, however, require manual intervention—specifically `redis-cli --cluster fix`—to drive the cluster back to a settled three-master state. The experiment therefore records the operator's correctness on the primary metric of data safety alongside a documented operational cost, rather than an unqualified endorsement (Redis Ltd., 2024b).

7.3 Comparative Analysis

Table 1 consolidates the two outcomes into a single before-and-after view. Read across its rows, the table isolates the one factor that is responsible for the difference between total data preservation and catastrophic loss: neither the underlying hardware, nor the dataset, nor the load profile, nor the resource budget differed between the two runs, so that the entire divergence is attributable to whether the scaling mechanism was aware of cluster topology. The native mechanism, which manages pod count but is structurally blind to hash slots, abandoned one quarter of the keyspace the instant it deleted a master, whereas the operator, which encodes the Redis-specific knowledge that a master must be drained before it is removed, preserved every slot and every key.

Table 1: Before-and-after comparison of the two scaling mechanisms for the one-master scale-in ($4 \rightarrow 3$). Slot-level figures are deterministic; the after-key figures are reported from the captured run.

Measurement	**Scenario A (HPA)**	**Scenario B (Operator)**
`cluster_state` (before)	`ok`	`ok`
`cluster_state` (after)	`fail`	`ok`
Slots covered (before)	$16\,384$	$16\,384$
Slots covered (after)	$12\,288$	$16\,384$
Keys stored (before)	5000	5000
Keys stored (after)	≈ 3750	5000
Keys lost	≈ 1250 (one quarter)	0
Read probe (errored / total)	$5/5$	$0/5$
Verdict	Data cliff	Graceful drain

This substantiates the thesis's central claim that topology-aware automation is a necessary condition for the safe elastic operation of a sharded, stateful workload, while at the same time qualifying that claim in an important respect. The operator is necessary, but it is not, on its own, sufficient for fully hands-off operation: its lack of self-healing leaves a residual class of interrupted-reshard failures that, at present, only a human

operator can resolve. The practical reading of these results is therefore not that the operator renders scaling effortless, but that it relocates the operator's burden from the unrecoverable (reconstructing lost data after a cliff) to the recoverable (completing a stalled but data-safe migration)—a decisive qualitative improvement even where it is not a complete elimination of operational toil.

7.4 Threats to Validity

Several deliberate constraints bound the interpretation of these results. First, the experiment was conducted under quiescent conditions; this was an intentional control, adopted so that an interrupted reshard caused by live write pressure could not be mistaken for a deficiency of the mechanism, but it does mean that the operator's robustness under concurrent load is not characterised here and is properly left to the scale-out experiments and to future work. Second, the HPA scenario was formed without replicas, which represents the starkest rather than the average case; this choice isolates the scaling mechanism cleanly and should be read as a demonstration that redundancy is no substitute for slot migration, not as a claim about every conceivable HPA topology, since replicas would delay rather than avert the cliff. Third, although the slot-level outcomes are deterministic—a direct consequence of the equal, 4096-slot partitions produced at cluster formation—the exact number of lost keys depends on the hash distribution of the specific dataset and is consequently reported from the captured run rather than derived analytically. Finally, the comparison concerns a single one-master transition; while the conclusions extend naturally to any individual planned scale-in, the experiment does not characterise repeated or multi-master contractions, which would compound the operator's resharding workload and are likewise reserved for future investigation.

8 Conclusion

This thesis set out from a deceptively simple observation—that a Kubernetes pod is a unit of scheduling whereas a Redis shard is a unit of data ownership, and that the two coincide only for as long as the cluster's topology remains undisturbed—and asked what happens to a sharded, stateful workload when that topology is disturbed in the most consequential way, by removing a slot-owning master during a planned scale-in. Through a deliberately minimal and fully controlled experiment, it has shown that the answer is governed not by how a scaling decision is timed, nor by whether redundancy is present, nor by the load the cluster happens to be under, but by a single architectural property of the mechanism that carries the removal out: whether or not that mechanism is aware of the cluster's slot topology.

8.1 Revisiting the Research Question

The central question—whether a planned, one-master scale-in of a sharded Redis Cluster preserves all $16\,384$ hash slots together with every stored key, and to what extent the answer depends on the scaling mechanism—admits a clear and sharply divided answer. When the removal was performed by the native StatefulSet contraction, which is precisely the action a Horizontal Pod Autoscaler issues on scale-down, the cluster suffered an immediate and irreversible *data cliff*: the departing master's slots, neither drained nor resharded, were abandoned the instant its pod terminated, slot coverage fell below $16\,384$, the cluster state transitioned to `fail`, roughly a quarter of the keyspace became permanently unreachable, and—under the default `cluster-require-full-coverage` policy—every read was refused with a `CLUSTERDOWN` error. When the same transition was requested of the Opstree Redis Operator, the outcome was the exact opposite: the operator drained the departing master, migrating its slots to the survivors before removing the pod, so that full coverage was held throughout and not a single key was lost. The hypothesis advanced in Section 2—that topology-aware operation is a precondition for data-safe scale-in—is therefore confirmed.

Two features of this result give it particular force. First, the loss in the native case occurred under perfectly quiescent conditions, with no client traffic of any kind, which establishes the cliff as a *structural* consequence of the mechanism rather than an artefact of write pressure, contention, or timing. Second, the divergence between total preservation and catastrophic loss was produced while every other variable—the hardware, the dataset, the resource budget, and the absence of load—was held identical, which isolates topology-awareness as the sole cause. Redundancy is no substitute: a replica would have delayed the cliff by one promotion but could not have prevented it, because a planned scale-in fundamentally requires the migration of slots that no native primitive performs.

8.2 Contributions

The principal contributions of this thesis may be stated concisely. It demonstrates, in isolation and with a deterministic and reproducible method, that the native, topology-blind scaling mechanism inflicts structural and irreversible data loss on the scale-in of a sharded Redis Cluster, and that a topology-aware operator preserves the keyspace in full under identical conditions (Redis, n.d.; OpsTree Solutions, n.d.). In doing so it establishes that the determining factor for data safety is the scaling *mechanism* rather than the scaling *trigger*, thereby reframing a question that the existing literature has approached almost exclusively through the lens of reactive-versus-proactive timing. Methodologically, it contributes a minimal experimental design—direct dataset population, a single quiescent one-master transition, and measurement through the cluster's own slot and key telemetry—that strips away the confounds of a fuller pipeline and renders the mechanism the only variable. Finally, and importantly for the credibility of the result, it characterises the proposed solution honestly rather than uncritically: the operator is shown to be *necessary but not*, *on its own*, *sufficient* for fully hands-off elastic operation, because its lack of self-healing for an interrupted reshard can leave a slot in an open, mid-migration state that a human operator must settle by hand with `redis-cli --cluster fix`.

8.3 Reflection on Scope and Methodology

The strength of the foregoing result rests on the deliberate narrowness of the experiment that produced it, and it is worth reflecting on that scoping explicitly. The study was confined to the data-safety of a single scale-in under quiescent conditions, and this confinement was not an evasion of difficulty but a direct response to one. An earlier, fuller pipeline—comprising an external load generator, an application entrypoint, and an event-driven autoscaling trigger—was found to confound the very measurement it was meant to support: under a sustained, write-heavy workload, a continuous stream of writes addressed to a slot that was mid-migration was the dominant cause of an interrupted reshard, and an interruption so caused would have been indistinguishable from a genuine defect of the scaling mechanism. Removing the load, the entrypoint, and the trigger, and writing the dataset directly, eliminated that confound and made the outcome a deterministic property of the mechanism alone. The load-induced interruption is not thereby dismissed; it is itself a finding, namely that write pressure rather than the mechanism is the governing factor in reshard interruption, and it is what justifies the controlled isolation.

It follows that certain elements present in the thesis's initial conception were, in the final analysis, deliberately reframed as out of scope rather than evaluated. The event-driven trigger layer in particular—the question of whether scaling decisions are taken reactively or proactively, and the role of application-level metrics in that decision—was set aside as orthogonal to the data-safety question, on the reasoning that no quality of trigger can rescue an unsafe removal mechanism and none is needed to vindicate a safe one. What remains, and what the thesis ultimately defends, is a tightly bounded but firmly established claim about the scaling mechanism itself. The remaining limitations

follow from the same boundary and are enumerated among the threats to validity in Section 7: the operator's robustness under concurrent load is not characterised here, the native cluster was formed without replicas so as to expose the cliff in its starkest form, and the comparison concerns a single one-master transition rather than a repeated or multi-master contraction.

8.4 Future Work

These boundaries map directly onto the most promising directions for further work. The most immediate is the reintroduction of live load under controlled conditions, in order to characterise how frequently the operator's reshard is interrupted under realistic write pressure and how that frequency scales with throughput—turning the confound identified above into an object of study in its own right. A natural second direction is to layer an event-driven trigger, such as the Kubernetes Event-Driven Autoscaling project, atop the topology-aware mechanism, so as to combine a proactive scaling decision with a data-safe scaling action and to evaluate the pairing against the reactive baseline. A third, and perhaps the most consequential for practice, is to close the self-healing gap that renders the operator necessary but not sufficient: automating the detection and repair of an interrupted reshard would move the operator from data-safe-but-supervised toward genuinely hands-off operation. Beyond these, the comparison invites extension to repeated and multi-master contractions, to clusters that enable replication and persistence, to substantially larger datasets, and to other operators and other sharded stores, so as to test the generality of the conclusion drawn here.

8.5 Concluding Remarks

The title of this thesis asserts that pods are not shards, and the experiment has given that assertion an empirical edge. Treating the removal of a stateful, slot-owning node as though it were the removal of an interchangeable, stateless replica does not merely degrade performance; it destroys data, structurally and irreversibly, and it does so the moment the pod is deleted. The remedy is not a faster or more anticipatory autoscaler but an autoscaling *mechanism* that understands what it is removing—one that drains before it deletes. Such topology-aware operation, as embodied here by a Kubernetes operator, is therefore a necessary foundation for the safe elastic operation of sharded, stateful workloads; the remaining distance to fully autonomous operation is measured precisely by the operational toil that the operator, as it stands today, still leaves to a human hand.

Anhang

Hier steht der Anhang.

Glossar

Hier steht das Glossar.

Literaturverzeichnis

Amazon Web Services. (2022). *Container build lens — AWS well-architected framework: Design principles* [Accessed: 2026-01-24]. https://docs.aws.amazon.com/pdfs/wellarchitected/latest/container-build-lens/container-build-lens.pdf#design-principles

Cloud Native Computing Foundation. (2023). *CNCF announces graduation of KEDA* [Accessed: 2026-01-24]. https://www.cncf.io/announcements/2023/08/22/cloud-native-computing-foundation-announces-graduation-of-kubernetes-autoscaler-keda/

Cloud Native Computing Foundation. (2024). *Managing large-scale Redis clusters on Kubernetes with an operator* [Accessed: 2026-01-24]. https://www.cncf.io/blog/2024/12/17/managing-large-scale-redis-clusters-on-kubernetes-with-an-operator-kuaishous-approach/

CNCF TAG App Delivery. (2023). *CNCF operator white paper* [Accessed: 2026-01-24]. https://tag-app-delivery.cncf.io/whitepapers/operator/

Docker Inc. (2016). *Containers are not VMs* [Accessed: 2026-01-24]. https://www.docker.com/blog/containers-are-not-vms/

Google Cloud. (2023). *Zero-downtime scaling in Memorystore for Redis cluster* [Accessed: 2026-01-24]. https://cloud.google.com/blog/products/databases/zero-downtime-scaling-in-memorystore-for-redis-cluster

Grafana Labs. (2024). *Grafana — the open and composable observability platform* [Accessed: 2026-01-24]. https://github.com/grafana/grafana

Helm Project. (2024). *Using Helm* [Accessed: 2026-01-24]. https://helm.sh/docs/intro/using_helm/

IBM. (2024). *The history of kubernetes* [Accessed: 2026-01-24]. https://www.ibm.com/think/topics/kubernetes-history

K3s Project. (2024). *K3s — lightweight kubernetes* [Accessed: 2026-01-24]. https://docs.k3s.io/

KEDA Project. (2024a). *KEDA — kubernetes event-driven autoscaling* [Accessed: 2026-01-24]. https://keda.sh/

KEDA Project. (2024b). *KEDA — scaling deployments, StatefulSets & custom resources* [Accessed: 2026-01-24]. https://keda.sh/docs/2.19/concepts/scaling-deployments/

Kubernetes Documentation. (2024a). *Autoscaling workloads* [Accessed: 2026-01-24]. https://kubernetes.io/docs/concepts/workloads/autoscaling/

Kubernetes Documentation. (2024b). *Kubernetes overview* [Accessed: 2026-01-24]. https://kubernetes.io/docs/concepts/overview/

Kubernetes Documentation. (2024c). *Operator pattern* [Accessed: 2026-01-24]. https://kubernetes.io/docs/concepts/extend-kubernetes/operator/

Kubernetes Documentation. (2024d). *Statefulsets* [Accessed: 2026-01-24]. https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/

Mell, P., & Grance, T. (2011). *The NIST definition of cloud computing* (tech. rep. No. Special Publication 800-145) (Accessed: 2026-01-24). National Institute of Standards and Technology. https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-145.pdf

OpsTree Solutions. (n.d.). Redis-operator: A Golang based Redis operator for Kubernetes [Accessed: 2026-06-06].

Opstree Solutions. (2023). Redis operator for kubernetes. https://ot-container-kit.github.io/redis-operator/

Prometheus Authors. (2024). *Prometheus overview* [Accessed: 2026-01-24]. https://prometheus.io/docs/introduction/overview/

Prometheus Community. (2024). *Kube-prometheus-stack Helm chart* [Accessed: 2026-01-24]. https://artifacthub.io/packages/helm/prometheus-community/kube-prometheus-stack

Redis. (n.d.). *Scale with redis cluster* [Accessed: 2026-06-06]. https://redis.io/docs/latest/operate/oss_and_stack/management/scaling/

Redis Ltd. (2024a). *Redis — real-time data for agents & apps* [Accessed: 2026-01-24]. https://redis.io/

Redis Ltd. (2024b). Redis cluster administration. https://redis.io/docs/management/scaling/

Redis Ltd. (2024c). *Redis cluster specification* [Accessed: 2026-01-24]. https://redis.io/docs/latest/operate/oss_and_stack/reference/cluster-spec/

Redis Ltd. (2024d). Redis cluster specification. https://redis.io/docs/reference/cluster-spec/

The Kubernetes Authors. (n.d.-a). *Horizontal pod autoscaling* [Accessed: 2026-06-06]. https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/

The Kubernetes Authors. (n.d.-b). *Operator pattern* [Accessed: 2026-06-06]. https://kubernetes.io/docs/concepts/extend-kubernetes/operator/

The Kubernetes Authors. (n.d.-c). *Statefulsets* [Accessed: 2026-06-06]. https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/

The Kubernetes Authors. (2024). Statefulsets. https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/

Thoras. (2024). *Predictive horizontal pod autoscaling* [Accessed: 2026-01-24]. https://docs.thoras.ai/guides/hpa

Wanigasooriya, C., & Ekanayake, I. (2026). NimbusGuard: A novel framework for proactive Kubernetes autoscaling using deep Q-networks [Accessed: 2026-01-24]. *arXiv preprint arXiv:2604.11017*. https://arxiv.org/html/2604.11017v1

Stichwortverzeichnis

Hier steht das Stichwortverzeichnis.

Anlagen

Hier sind die Anlagen.